

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Tomás San Martín

8 de septiembre de 2026

16:15

1. Introducción

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

Escriba una introducción breve que presente los dos problemas estudiados (ordenamiento de arreglos unidimensionales y multiplicación de matrices cuadradas) y el objetivo del análisis experimental: relacionar el comportamiento práctico de los algoritmos con su complejidad teórica.

2. Introducción

La notación asintótica acota el crecimiento de un algoritmo, pero no informa sobre constantes ocultas, jerarquía de memoria ni el efecto del tipo de entrada. Este trabajo estudia experimentalmente cuatro algoritmos de ordenamiento y dos de multiplicación de matrices cuadradas, midiendo tiempo y memoria sobre 72 casos por problema (Anexo A), y contrasta lo observado con las cotas teóricas $O(n \log n)$, $O(n^3)$ y $O(n^{\log_2 7})$. El código, los datos y los scripts de medición y graficado se incluyen en el directorio `code/`.

3. Entorno experimental

4. Marco experimental

Equipo: Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz, 24 GB RAM, Windows 11 con WSL2 (Ubuntu); compilador g++ [versión] con flags `-O2 -std=c++17` (en todos los algoritmos). Los casos de prueba se generaron con los scripts provistos por el curso: ordenamiento con $N = \{10, 10^3, 10^5, 10^7\}$, $T = \{\text{ascendente, descendente, aleatorio}\}$, $D = \{D1, D7\}$, $M = \{a, b, c\}$; matrices con $N = \{2^4, 2^6, 2^8, 2^{10}\}$, $T = \{\text{dispersa, diagonal, densa}\}$, $D = \{D0, D10\}$, $M = \{a, b, c\}$ (72 casos por problema). Como limitaciones en el desarrollo de la tarea tenemos efectos de caché al momento de comparar teórico y experimentalmente los algoritmos, y la resolución del muestreo de memoria.

5. Resultados y Análisis

6. Resultados y discusión

6.1. Ordenamiento

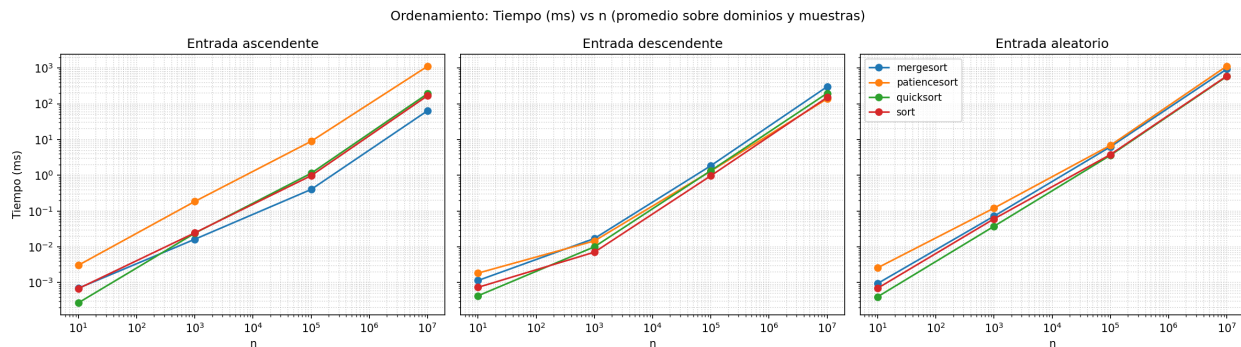


Figura 1: Tiempo de ordenamiento vs n (escala log-log), promedio sobre dominios y muestras.

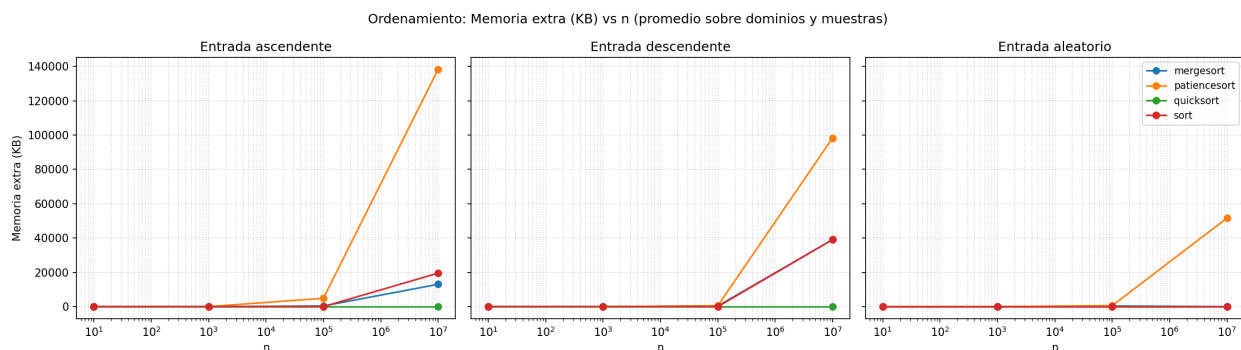


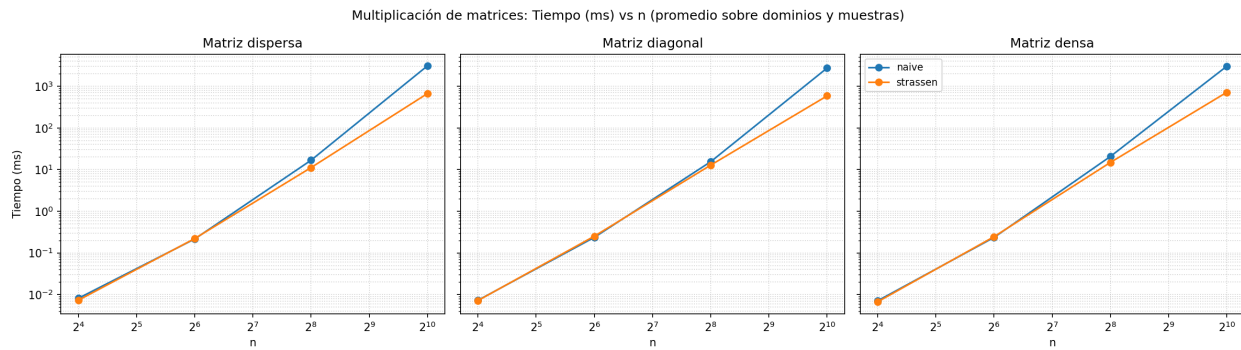
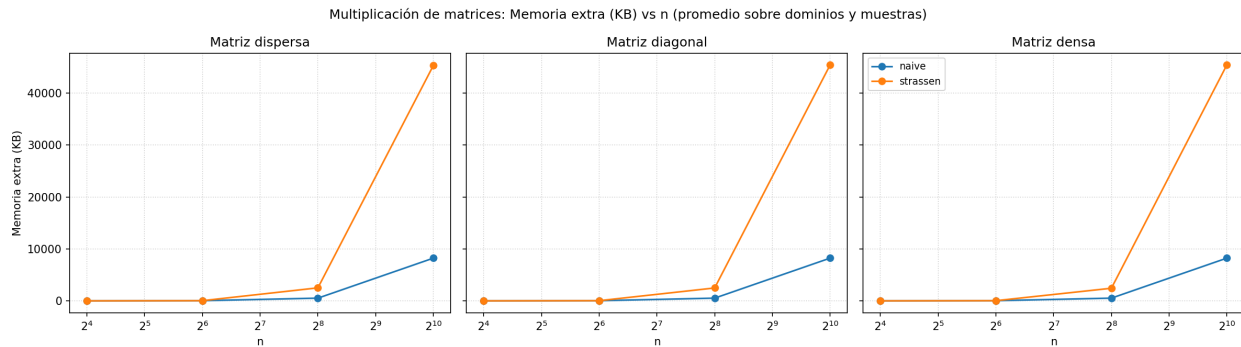
Figura 2: Memoria extra de ordenamiento vs n .

Cuadro 1: Tiempo (ms), entrada aleatoria, dominio D7, muestra a.

n	merge	quick	patience	std::sort
10^5	7.98	5.17	10.26	5.42
10^7	1247.82	867.64	1870.45	862.97

En escala log-log las curvas presentan pendiente cercana a 1, coherente con $O(n \log n)$. `std::sort` y quick sort dominan en entrada aleatoria; merge sort es el más rápido en entrada ascendente gracias a la fusión condicional, mientras que patience sort se degrada notablemente en ese caso porque genera aproximadamente n pilas. En memoria, merge y patience crecen lineales ($O(n)$: buffer y pilas más arreglo de salida), en tanto que quick sort y `std::sort` permanecen planos: su espacio extra es pila de recursión $O(\log n)$, no heap.

6.2. Multiplicación de matrices

Figura 3: Tiempo de multiplicación vs n (log-log).Figura 4: Memoria extra de multiplicación vs n .

Naive exhibe pendiente ≈ 3 ($O(n^3)$) y Strassen $\approx 2,81$ ($O(n^{\log_2 7})$). El cruce aparece entre 2^8 y 2^{10} : la constante oculta de Strassen (siete productos recursivos más las sumas de matrices) solo se amortiza en tamaños grandes. En memoria, naive asigna únicamente la matriz resultado (≈ 8 MB en

Cuadro 2: Tiempo (ms), matriz densa, dominio D10, muestra a.

n	naive	strassen
2^8	16.62	13.24
2^{10}	3296.23	712.13

$n = 1024$), mientras Strassen alcanza ≈ 45 MB por los temporales recursivos: el divide-and-conquer compra tiempo con espacio.

7. Conclusiones

8. Conclusión

El comportamiento práctico reproduce las cotas teóricas en el régimen asintótico, pero la constante oculta, el tipo de entrada y las decisiones de implementación (mediana de tres, fusión con min-heap, umbral de corte en Strassen) determinan qué algoritmo conviene en cada rango de n . El análisis experimental es, por tanto, el complemento necesario de la notación asintótica para seleccionar algoritmos en la práctica, ya que también entran en juego la jerarquía de memoria y la eficiencia de caché, que no se reflejan en la complejidad teórica, y que afectan en lo que podríamos llegar a esperar.